Introdução Html + Java Server Page

Conteudista: Prof. Me. Alexander Gobbato Paulino Albuquerque
Revisão Textual: Prof. Claudio Brites

Objetivos da Unidade:

- Abordar o conceito de cliente servidor, Html e *Java* para *Web*;
- Adquirir conhecimento para criar um projeto simples utilizando os conceitos abordados.

Material Teórico

Material Complementar

Referências

Material Teórico

Introdução

Antes de começarmos a colocar a mão na massa com instalações, configurações e tudo mais, vamos entender e falar de alguns conceitos que são de extrema importância.

Em um mundo onde tudo está conectado, você já ouviu falar em arquitetura *stand-alone*?

Stand-Alone

Caracteriza-se pelo uso de uma única estação de trabalho autossuficiente, na qual se centraliza a interface com o usuário, o processamento e o armazenamento de dados. Não há utilização de rede de comunicação.

Um computador ainda que ligado numa rede local ou com conexão a Internet poderá executar aplicativos *stand-alone*, que executarão no próprio computador, sem interagir direta ou indiretamente com outros computadores, processadores ou fontes de dados.

Tem uma outra arquitetura que é muito comum no nosso cotidiano, a arquitetura Cliente/Servidor.

O ponto principal em uma arquitetura Cliente-Servidor está na presença de elementos centralizadores de dados, de *hardware* ou de processamento (servidores), em geral bem mais robustos que os outros elementos que são deles dependentes (clientes).

Veja o modelo abaixo de uma arquitetura Cliente-Servidor só que para a *web*.

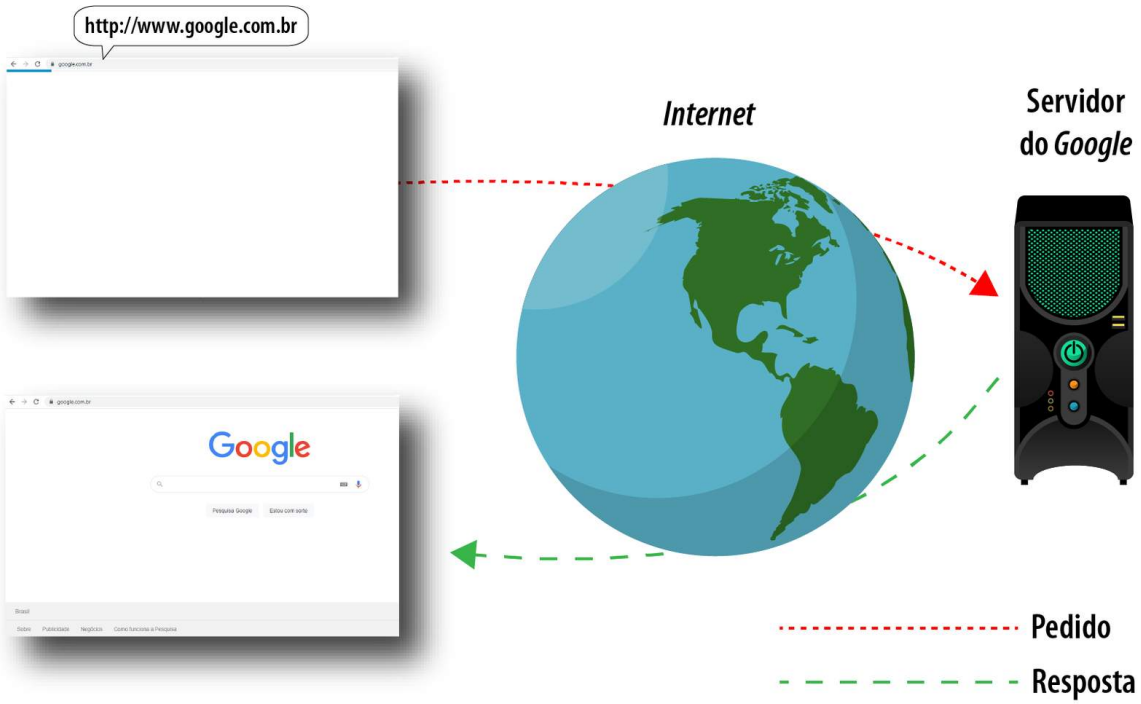


Figura 1 – Representação da arquitetura
Fonte: Adaptada de Freepik

Arquiteturas Cliente/Servidor de Duas e Três Camadas

Uma arquitetura cliente/servidor poderá ser classificada pela estrutura e funcionamento em:

Arquitetura Cliente/Servidor de Duas Camadas

- Interface do sistema de usuário + algum processamento;
- Processamento principal + gerenciamento dos Bancos de Dados.

Nesse modelo, a interface do sistema é usualmente gerada e colocada na máquina do cliente e os serviços de processamento e gerenciamento dos bancos de dados são usualmente colocados em um servidor que, em geral, é uma máquina mais potente, capaz de atender ou servir vários clientes. Sempre que houver atualização na interface os clientes precisarão instalá-la.

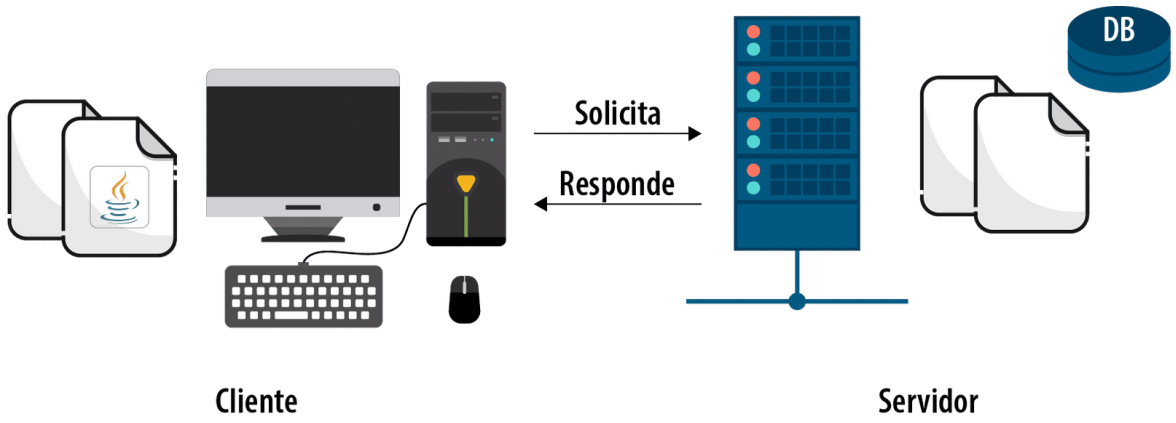


Figura 2 – Arquitetura cliente/servidor
Fonte: Adaptada de Freepik

Arquitetura Cliente/Servidor de Três Camadas ou Multicamadas

- Interface do sistema de usuário + algum processamento;
- Gerenciamento de processos;
- Gerenciamento dos Bancos de Dados.

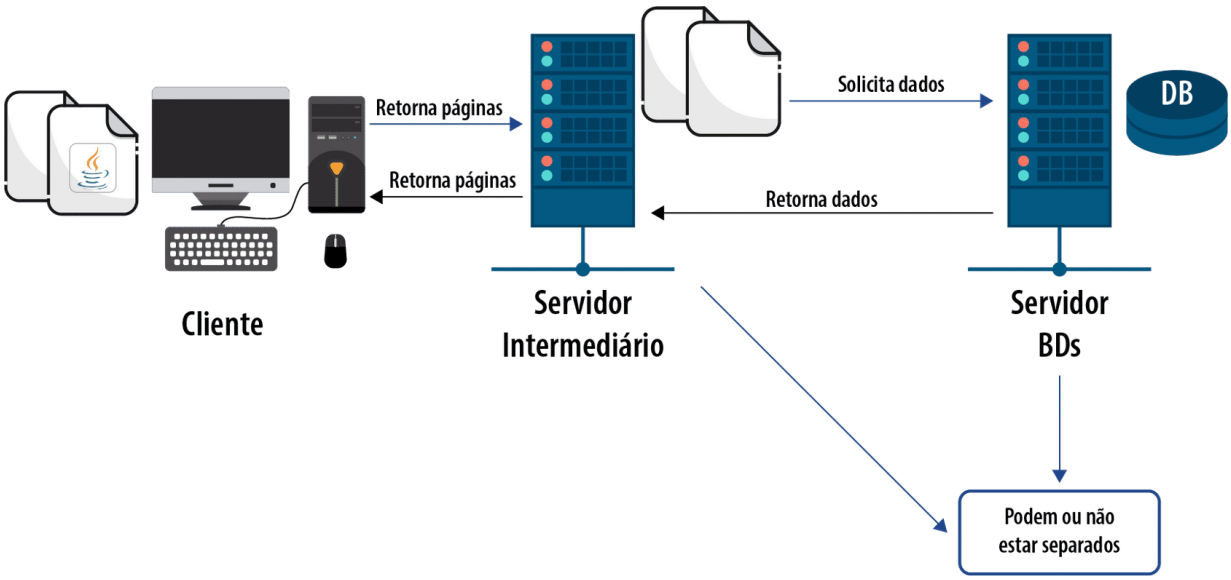


Figura 3 - Arquitetura 3 camadas
Fonte: Adaptada de Freepik

O Protocolo HTTP

O serviço *HTTP* (*HyperText Transfer Protocol* – protocolo de transferência de hipertexto) funciona de forma semelhante ao serviço *FTP* (*File Transfer Protocol* – protocolo de transferência de arquivos).

Tanto o HTTP quanto o FTP usam um sistema de arquivos virtual, que pode ter uma hierarquia própria e diferente do sistema de arquivos real das máquinas servidoras.

Veja a seguir.

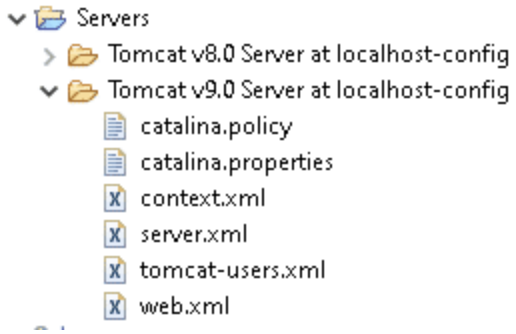


Figura 4 – Estrutura *Eclipse* (*Server*)
Fonte: Acervo do conteudista

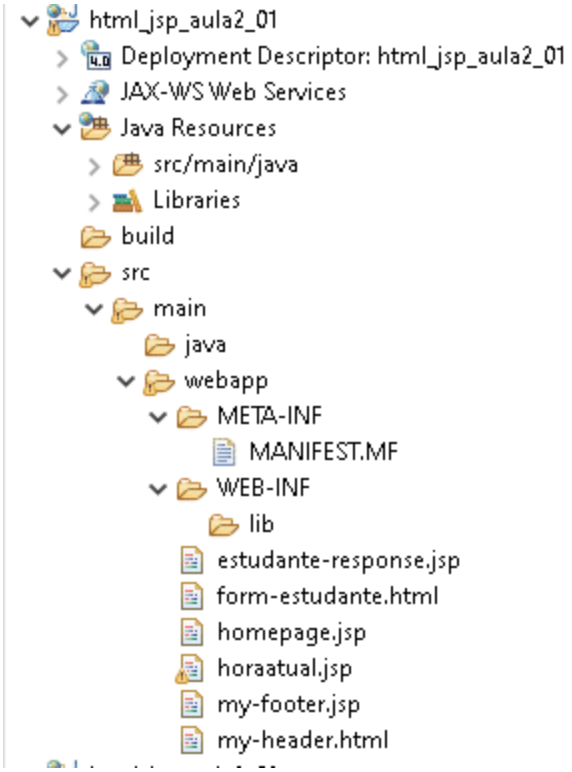


Figura 5 – Estrutura Aplicação
Fonte: Acervo do conteudista

Detalhando o Protocolo HTTP

Diferentemente do FTP, o protocolo HTTP não mantém uma sessão aberta entre transferências. Cada operação consiste em um único envio de requisição pelo cliente, seguido de uma única resposta. Ou seja, acontece uma conexão, um envio de requisição, um recebimento de resposta e o fim da conexão.

Não há persistência de informações entre as transações (não é possível manter o estado de variáveis entre páginas).

HTTP é, portanto, um protocolo que não mantém estado nem conexão.

A comunicação entre o *browser* e o servidor ocorre através de requisições e respostas HTTP.

Cada requisição contém uma linha com um método HTTP detalhando como e o que deve ser pedido ao servidor, seguido por um cabeçalho com informações adicionais sobre a requisição (ex. MIME *type*).

Métodos HTTP

HTTP define um conjunto de mensagens chamadas de métodos de requisição HTTP que são usadas pelo cliente para enviar uma requisição ao servidor.

A versão HTTP 1.1 define vários métodos básicos, porém os mais conhecidos são o *GET* e o *POST*.

Um servidor *Web* típico recebe normalmente requisições *GET*, *HEAD* e *POST*, sendo a grande maioria requisições *GET*, *POST*.

Enviando Dados pelos Métodos *GET* e *POST*

Os métodos *GET* e *POST* são usados pelo cliente para enviar dados ao servidor durante uma requisição.

Dados enviados via *GET* são incluídos em uma *query string* que é anexada à URL da solicitação (tamanho máximo da URL: 2048 caracteres).

POST é indicado para enviar grandes quantidades de dados ao servidor, já que não é limitado como o *Query String* usado para enviar informações via *GET*.

Anexos devem ser enviados via *POST*.

Método *GET*

Observe que a *query string* (pares nome/valor) é enviada na URL de uma solicitação *GET*:

```
/test/demo_form.asp?name1=value1&name2=value2
```

Método *POST*

Observe que a *query string* (pares nome/valor) é enviada no corpo da mensagem de uma solicitação HTTP com método *POST*:

```
POST/test/demo_form.aspHTTP/1.1
Host: w3schools.com
name1=value1&name2=value2
```

Alguns Códigos de Repostas a uma Solicitação HTTP

- ***OK* 200:** a requisição foi atendida;
- ***No Response* 204:** o servidor recebeu a requisição, mas não há nada a ser retornado;
- ***Bad request* 400:** a sintaxe do pedido era inadequada ou impossível de ser satisfeita;
- ***Unauthorized* 401:** o cliente não tem autorização para obter a informação;
- ***Forbidden* 403:** o objeto não pode ser enviado, nem mesmo com autorização;
- ***Not found* 404:** página não encontrada. É, sem dúvida, a mensagem mais comum;
- ***Internal Error* 500:** erro interno;
- ***Not implemented* 501:** recurso não implementado.

JSP (JavaServer Pages)

JSP é híbrido, pois suporta dois estilos diferentes para adicionar conteúdo dinâmico às páginas HTML, através de códigos *Java* e *tags* incorporados nas próprias páginas.

```
<html>
<body>
    <%
        String hello="Hello, world!<br>";
        for(int i=0; i<20; i+=2)
           out.println(hello);
    %>
</body>
</html>
<html>
<body>
    <%
        String hello="Hello, world!<br>";
        for(int i=0; i<20; i+=2)
           out.println(hello);
    %>
</body>
</html>
```

Figura 6 – Exemplo código
Fonte: Acervo do conteudista

Agora chegou a hora de colocarmos a mão na massa ou como disse vamos para o *on the hands*!

Primeiro ponto para começarmos é baixar, instalar e configurar o ambiente de desenvolvimento. Para isso, realize o *download* do material a seguir para te auxiliar.

PDF

Instalando e configurando o Eclipse e Tomcat.pdf
857.4 KB

Conceitos Iniciais

Java Server Pages (*JSP*) é uma tecnologia baseada em *Java*, que simplifica o processo de engenharia de *websites*.

Uma página JSP é um arquivo-texto, geralmente com extensão .jsp, podendo conter trechos (*scriptlets*) de código *Java* puro, *tags* específicas JSP, "expressões" *Java* e outros elementos.

A partir da página JSP, o contêiner JSP no servidor *Web* gerará um *Servlet*.

> “O *Java* EE é um ambiente independente da plataforma, centrado em Java que cria e implementa aplicativos corporativos baseados na *Web*. O *Java* EE inclui muitos componentes do *Java Standard Edition* (*Java SE*). A plataforma *Java* EE consiste em um conjunto de servidores, APIs e protocolos que fornecem a funcionalidade para desenvolver aplicativos multicamadas com base na *Web*.”
>
> - ***JAVA*, s. d.**

Site

Java

Clique no botão para conferir o conteúdo.

ACESSE

Leitura

Differences Between Java EE and Java SE

Clique no botão para conferir o conteúdo.

ACESSE

Elementos Básicos de JSP

- *Tags* para comentários;
- **Diretivas:**
  - Diretivas de página;
  - Diretivas para acesso a bibliotecas de *tags*.
- **"Elementos de ação" ou "*tags* de ação" de JSP:**

    - *Tags* relacionadas com bibliotecas de *tags*;
    - *Tags* "de ação padrão" (ou *standard action elements*, que começam com jsp).

- *Scriptlets* (código *Java* dentro da página);
- Expressões;
- Declarações.

Características Básicas do JSP

- ***Tag* para comentários:** serve para inserirmos dentro do HTML comentários que podem ajudar na documentação. Exemplo:

```
<%-
Esse é um exemplo de comentário de uma
ou várias linhas
--%>
```

- ***Tag* para *scriptlets*:** servem para inserirmos código *java* dentro do hmtl. Exemplo:

```
<%
   //código Java.....
%>
```

- ***Tag* para expressões:** servem para retornar um elemento ou um valor. Exemplo:

```
<%=  ............  %>
```

- ***Tag* para diretivas de páginas:** servem para incluir informações necessárias para processar uma classe *servlet* que foi gerada de um JSP. Exemplo:

```
<%@ page import ="java.util.* , java.sql.*"  %>

<%@ page errorPage="paginaErro.jsp" %>
```

- ***Tag* para diretivas a bibliotecas:** servem para acessar ou ter acesso as bibliotecas (JSTL). Exemplo:

```
<%@ taglib prefix="c"  uri="http://java.sun.com/jsp/jstl/core" %>

<%@ taglib prefix="x" uri="http://java.sun.com/jsp/jstl/xml" %>

<%@ taglib prefix="fmt" uri="http://java.sun.com/jsp/jstl/fmt" %>

<%@ taglib prefix="sql" uri="http://java.sun.com/jsp/jstl/sql" %>
```

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Livros

Java para a *Web* com *Servlets*, JSP e EJB

KURNIAWAN, B. ***Java* para a *web* com *servlets*, JSP e EJB**. Rio de Janeiro: Ciência Moderna, 2002.

Ajax, *Rich Internet Applications* e Desenvolvimento *Web* para Programadores

DEITEL, P. J.; DEITEL, H. M. ***Ajax, rich internet applications* e desenvolvimento *web* para programadores**. Pearson 776 ISBN 9788576051619. (*e-book*)

Aprenda J2EE: com EJB, JSP, *Servlets*, JNDI, JDBC e XML

BOND, M. *et al.* **Aprenda J2EE**: com EJB, JSP, *Servlets*, JNDI, JDBC e XML. São Paulo: Pearson Education do Brasil, 2003. (*e-book*)

Projetando *Websites*

NIELSEN, J. **Projetando *websites***. Rio de Janeiro: Campus, Elsevier, 2000

Referências

FREEMAN, E.; FREEMAN, E. **Use a Cabeça! Html Com Css e Xhtml.** São Paulo: Alta Books, 2008.

GUIMARÃES, F. G. ***Eclipse* com *Tomcat* e *Struts***. Grupo de usuários Java, 2019. Disponível em: <https://docplayer.com.br/125752955-Eclipse-com-tomcat-e-struts.html>. Acesso em: 02/06/2022